Foundations of Data Science for Biologists

Indexing, filtering, and grouping

BIO 724D

2025-SEP-16

Instructors: Greg Wray, Paul Magwene

Revisiting indexing

Revisiting indexing

Indexing = mechanism to refer to a specific subset of a data object

Square bracket `df[x]` or `df['x']`

Integer or string argument; returns an *object* (usually a list)

Double square bracket `df[[x]]`

Integer argument; returns *values* (atomic vector if indexing a data frame)

Dollar sign `df$x`

String argument; returns *values* (item from list or column from data frame)

Tidyverse `verb(df, x)`

String argument; typically returns an *object* (list or data frame)

Take-away lessons about indexing

All four methods of indexing are useful and an essential part of basic R proficiency

Make your code robust:

- Refer to columns by name
- Avoid using ranges (even when indexing with column names)
- Refer to rows by condition (or use unique IDs); avoid numerical indexing

Remember to input column names in different ways for different methods:

- Use quotation marks for `obj[x]` and `obj[[x]]`
- Do not use quotation marks for `obj$` and `verb[obj, x]`

Remember that different methods return data in different forms:

- `obj[x]` and `verb[obj, x]` return lists
- `obj[[x]]` and `obj$x` return atomic vectors

Passing information in pipes

Many common functions work well in pipes

- Usually works if the first argument refers to the data
- True for most Tidyverse functions: `select()`, `filter()`, `ggplot()`, etc.
- True for many base R functions: `length()`, `mean()`, `min()`, etc.
- However, not true for all common functions: `lm()`

Using functions in pipes:

- Using a function by itself: `func(df, x)`
- Using a function in a pipe: `funcA(df, x) |> funcB(y) |> funcC(z)`
- The name of the data frame is only needed in the first step

Intermediate filtering

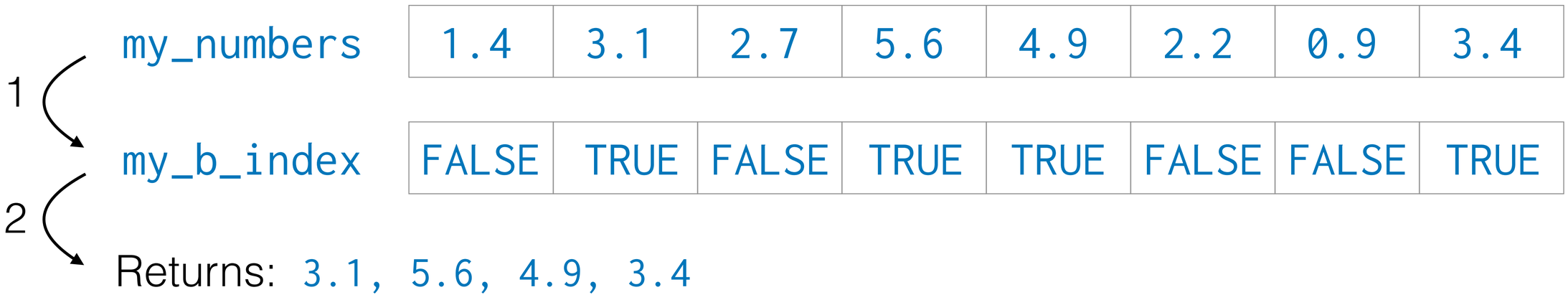
How does Boolean indexing work?

Step 1: create an index column based on a data column and a condition

```
my_df <- mutate(my_df, my_b_index = my_numbers > 3.0)
```

Step 2: filter based on TRUE/FALSE values

```
my_df |> select(my_numbers) |> filter(my_b_index)
```



Why use Boolean indexing?

Code is more readable: apply the condition once and then use it repeatedly

Updating the condition only has to be done once in your code

You can create multiple Boolean index columns for different conditions

Filtering on a Boolean index is *much* faster than applying the condition again

Introducing factors

Factors are categorical data vectors (homogenous, but different from atomic vectors)

- Categories are variables with a defined set of possible values, usually just a few

In R, categories are called **levels**

- By default, levels are not ordered (e.g., herbivore, carnivore, omnivore)
- Optionally, levels can be ordered (e.g., egg, larva, pupa, adult)

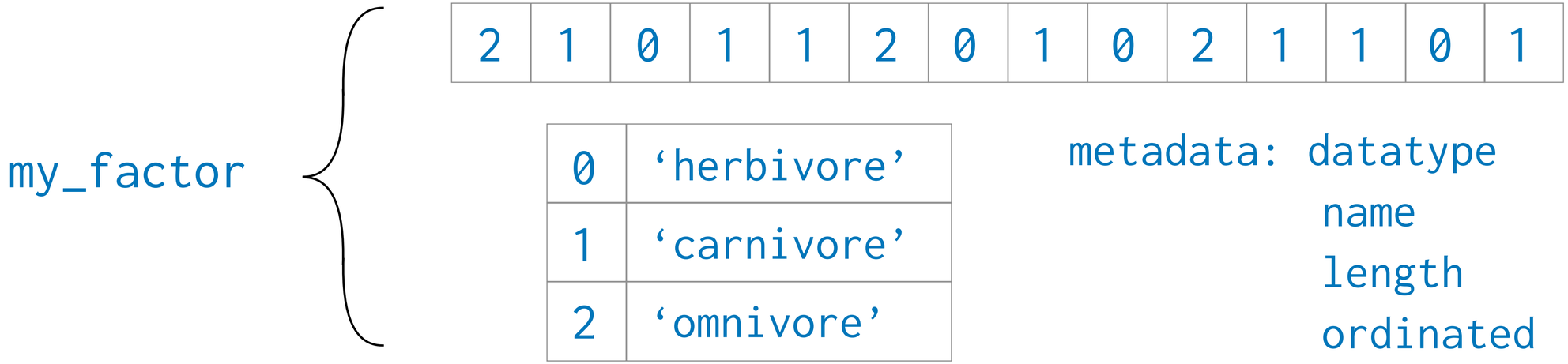
Why use factors?

- Memory-efficient: occupies less space than a character vector
- Computationally efficient: sort, group_by, and other operations are *much* faster
- Minimizes data entry errors: reports any non-standard values
- Required for some functions and 3rd-party packages

How do factors work?

Factors are structures with two data components, plus metadata

- Integer vector that stores the data
- Look-up table that maps integers to strings (optionally ordinated)



Factors provide the best of both worlds

- For humans: displayed as strings, thus simple to understand
- For computer: manipulated as integers, thus very memory- and processor-efficient